

# CTS SRE Day 10,11,12

Infrastructure performance & troubleshooting 1,2,3

**EMPOWERING** High Performance  
Technology Teams

# Agenda for the Day

- ✓ Understand **Infrastructure as Code (IaC)** with Terraform for automated provisioning.
- ✓ Implement **Application Performance Monitoring (APM)** for backend profiling.
- ✓ Utilize **Real User Monitoring (RUM)** to analyze frontend performance.
- ✓ Perform **transaction tracing** using Jaeger/OpenTelemetry/AWS for distributed systems.
- ✓ Correlate **APM, RUM, and Infrastructure Metrics** to troubleshoot performance issues.

# Introduction to Infrastructure Performance Monitoring

## • Why Performance Monitoring Matters?

- Ensuring **reliability, scalability, and efficiency** in cloud environments.
- Role of **APM, RUM, and IaC** in performance optimization.
- Tools: **Splunk, AWS CloudWatch, Datadog, ELK, Grafana, OpenTelemetry**

# Infrastructure as Code (IaC) Using Terraform

 **Objective:** Automate infrastructure provisioning for scalable monitoring solutions.

 **Key Topics:**

- ✓ **What is Terraform**, and why use it for infrastructure deployment?
- ✓ Writing **Terraform configurations** to provision AWS/GCP monitoring stacks.
- ✓ Deploying **CloudWatch dashboards, logging agents, and synthetic monitoring tools** using Terraform.
- ✓ Best practices for **scalability, security, and cost optimization**.

 **Hands-On Activity:**

- Write a **Terraform script** to deploy an EC2 instance with CloudWatch monitoring enabled.

# Application Performance Monitoring (APM) and Backend Profiling

**Objective:** Track and analyze application performance using **APM** tools.

## ◆ Key Topics:

- ✓ **APM Setup:** Configure **Datadog**, **New Relic**, or **AWS X-Ray** for API and database monitoring.
- ✓ **Backend Profiling:** Capture CPU, memory, and response time metrics.
- ✓ **Logging & Correlation:** Integrating logs from **Splunk/ELK** with APM data.
- ✓ **Synthetic Monitoring:** Simulating transactions to validate SLA performance.

# Real User Monitoring (RUM) for Frontend Performance

 **Objective:** Monitor frontend user experience with RUM tools.

 **Key Topics:**

- ✓ **Google Lighthouse:** Tracking page load times, user interactions, and frontend errors.
- ✓ **RUM Metrics:** Time to First Byte (TTFB), First Contentful Paint (FCP), Largest Contentful Paint (LCP).
- ✓ **Geolocation-based Performance Analysis:** Monitoring application responsiveness for different regions.
- ✓ **Correlating RUM and APM Data:** Detecting latency issues from frontend to backend.

# Transaction Traceability in Distributed Systems

 **Objective:** Track transactions across microservices for debugging and troubleshooting.

## Key Topics:

- ✓ **Jaeger/OpenTelemetry/AWS:** Implementing request tracing in a distributed microservices setup.
- ✓ **Transaction Flow Visualization:** Identify slow or failing requests.
- ✓ **Bottleneck Detection:** Analyze database query performance and inter-service latency.

# Auto-scaling SelfHealing Automation using Terraform Script

-  **Objective:** Deploy an automated **auto-scaling web server** cluster using Terraform
-  **Key Topics:**
  - ✓ **AWS Service/Lambda/Locust:** Use of aws service to automate auto-healing and Simulate traffic spikes using **Locust**.
  - ✓ **User Request and Response Time:** Identify request count **and** response time latency.
  - ✓ **Failure Detection:** Handle automated recovery from failure using lambda service.

# Correlating APM, RUM, and Infrastructure Metrics

-  **Objective:** Combine performance monitoring data across **infrastructure, applications, and users.**
-  **Key Topics:**
  -  **Grafana & Kibana Dashboards:** Unified monitoring across system components.
  -  **Log Correlation for Troubleshooting:** Map **frontend errors to backend failures.**
  -  **Root Cause Analysis (RCA):** Identify which component is causing a performance degradation.
  -  **Automated Alerts:** Set up alerts when latency, error rate, or resource utilization exceeds thresholds.
-  **Hands-On Activity:**
  - **Build a Grafana dashboard** integrating RUM, APM, and CloudWatch metrics.

## Hands on Exercises

---

# Labs

- [CTS SRE Cloudwatch lab](#)
- [https://github.com/AshishSharmaPrivate/Lab/blob/main/ec2\\_cloudwatch\\_script](https://github.com/AshishSharmaPrivate/Lab/blob/main/ec2_cloudwatch_script)
- [https://github.com/AshishSharmaPrivate/Lab/blob/main/Log\\_Correlation\\_for\\_Debugging\\_ELK\\_Stack](https://github.com/AshishSharmaPrivate/Lab/blob/main/Log_Correlation_for_Debugging_ELK_Stack)
- <https://github.com/AshishSharmaPrivate/Lab/blob/main/Application%20Performance%20Monitoring>
- <https://github.com/AshishSharmaPrivate/Lab/blob/main/Auto-scaling-SelfHealing-Automation-Infra-Monitoring>

# Overall Summary

-  Terraform automates monitoring infrastructure deployment.
-  APM tools help detect backend performance bottlenecks.
-  RUM tools provide real-time insights into frontend user experience.
-  Transaction tracing is critical for microservices debugging.
-  Correlation of APM, RUM, and infrastructure metrics enhances troubleshooting.
-  Q&A and Discussion

**Thank You!**  
**StackRoute**  
**(An NIIT Venture)**

